

RELÓGIO DIGITAL COM MULTIPLEXAÇÃO E INTERRUPÇÃO

Instituição: Universidade do Estado de Minas Gerais

Curso: Engenharia da Computação

Período: 7º Período

Disciplina: Microprocessador e Microcontrolador

Discente: Lucas Melo Monteiro Peixoto

10 de Maio de 2026 - Ituiutaba, MG

1. Introdução

O presente relatório documenta o desenvolvimento do firmware para um relógio digital em um microcontrolador da família PIC (8 bits) - PIC16F877A. O projeto engloba a utilização de periféricos internos fundamentais para sistemas embarcados: a configuração avançada do **Timer 2** para geração de uma base de tempo precisa e a técnica de **Multiplexação no tempo** para o controle de quatro displays de 7 segmentos. Adicionalmente, foram implementadas funcionalidades de *Start/Pause* e *Reset*.

2. Configuração do Timer 2

O Timer 2 é um temporizador/contador de 8 bits que opera com um registrador de período (PR2) e possui tanto *Prescaler* quanto *Postscaler*. Para o relógio digital, o Timer 2 foi configurado para gerar uma interrupção periódica (flag TMR2IF), que serve como o indicador para contagem de tempo.

A configuração dos registradores foi realizada da seguinte forma:

- **T2CON (Timer 2 Control Register):**
 - TMR2ON = 1: Habilita o funcionamento do Timer 2.
 - **Prescaler:** Configurado para **1:16** (T2CKPS1 = 1, T2CKPS0 = 0). Ele divide a frequência do ciclo de máquina antes de alimentar o contador do timer.
 - **Postscaler:** Configurado para **1:10** (TOUTPS[3:0] = 1001 em binário). Ele conta quantas vezes o Timer 2 atinge o valor de PR2 antes de efetivamente disparar a interrupção.
- **PR2 (Timer 2 Period Register):**
 - Carregado com o valor hexadecimal 0xF9, que equivale a **249** em decimal. O timer conta de 0 até PR2 (portanto, 250 contagens totais) e zera.
- **Interrupções:**
 - TMR2IE = 1 e TMR2IF = 0: Habilita a interrupção específica do Timer 2 e limpa a flag inicial.
 - PEIE = 1 e GIE = 1: Habilitam as interrupções de periféricos e a chave geral de interrupções.

3. Cálculo da Temporização de 1 Segundo

Para que o relógio funcione corretamente, precisamos que a variável de segundos seja

incrementada exatamente a cada 1 segundo. O microcontrolador está operando com um oscilador de **4 MHz**.

Passo 1: Encontrar o Ciclo de Máquina (T_{cy})

A arquitetura PIC divide a frequência do oscilador principal por 4 para executar uma instrução.

- $F_{osc} = 4 \text{ MHz}$
- $F_{cy} = \frac{4 \text{ MHz}}{4} = 1 \text{ MHz}$
- $T_{cy} = \frac{1}{1 \text{ MHz}} = 1 \text{ } \backslash \text{mus}$ (Cada incremento do timer leva 1 microssegundo).

Passo 2: Calcular o tempo de uma Interrupção (T_{int})

A fórmula para o tempo de estouro do Timer 2 é:

$$T_{int} = T_{cy} \times \text{Prescaler} \times (PR2 + 1) \times \text{Postscaler}$$

Substituindo com os valores configurados no software:

- $T_{int} = 1 \text{ } \backslash \text{mus} \times 16 \times (249 + 1) \times 10$
- $T_{int} = 1 \text{ } \backslash \text{mus} \times 16 \times 250 \times 10$
- $T_{int} = 40.000 \text{ } \backslash \text{mus}$
- $T_{int} = 40 \text{ ms}$

A interrupção do Timer 2 ocorre exatamente a cada **40 milissegundos**.

Passo 3: Atingir 1 Segundo

Sabendo que 1 segundo equivale a 1000 milissegundos, dividimos o tempo alvo pelo tempo da interrupção:

- $\text{Estouros} = \frac{1000 \text{ ms}}{40 \text{ ms}} = 25 \text{ estouros}$

Por isso, na Rotina de Serviço de Interrupção (ISR), foi criada uma variável contadora estouros. Somente quando a condição $\text{if}(\text{estouros} \geq 25)$ é satisfeita, as engrenagens do relógio digital (unidades e dezenas de segundos e minutos) são incrementadas.

4. Explicação da Multiplexação dos Displays

Para controlar 4 displays de 7 segmentos seriam necessários 28 pinos de I/O (4×7), o que inviabiliza o projeto em microcontroladores menores. A solução adotada foi a **Multiplexação no tempo**, utilizando o princípio da persistência retiniana do olho humano.

Arquitetura de Hardware:

- Todos os segmentos (A até G) dos 4 displays estão ligados em paralelo no **PORTD**.
- O acionamento de qual display deve ligar no momento é feito de forma individual (pinos comuns de anodo ou catodo) através dos pinos **RB4 a RB7** do **PORTB**.

Lógica de Software (multiplexar_display):

O software liga um display por vez em uma velocidade imperceptível ao olho humano. O ciclo ocorre da seguinte forma:

1. Zera o PORTB para apagar todos os displays (evitando o efeito fantasma ou *ghosting* de um número vazando para o outro).
2. Envia o dado do dígito correspondente ao display atual para o PORTD.
3. Liga o pino respectivo no PORTB (ex: RB4 = 1).
4. Aguarda um tempo de persistência de **1 milissegundo** (`__delay_ms(1)`).
5. Repete o processo para os próximos 3 displays.

Como cada display fica ligado por 1ms e o ciclo total leva 4ms (além de alguns microssegundos de processamento), a taxa de atualização completa do painel (Refresh Rate) é superior a **200 Hz** ($1/0,004s = 250Hz$). Como o olho humano não consegue distinguir cintilações (flicker) acima de 60 Hz, temos a ilusão de óptica de que os quatro displays estão brilhando continuamente ao mesmo tempo.

Para otimizar o processamento e garantir um brilho forte e constante, a matemática pesada (atualização das variáveis de unidade e dezena) foi totalmente transferida para dentro do evento de interrupção, deixando o laço principal (`while(1)`) exclusivamente dedicado à varredura e acionamento ininterrupto da multiplexação e leitura dos botões.

6. Vídeo da simulação em funcionamento

Captura do funcionamento do projeto, simulação feita no software PICSIMLAB com a placa virtual McLab2 e microcontrolador PIC16F877A: [VÍDEO](#)

7. Código do projeto

Repositório do github com os códigos associados ao projeto do relógio digital: [CÓDIGO](#)

8. Conclusão

A construção deste relógio digital consolidou conceitos cruciais de sistemas de tempo real. O uso do Timer 2 demonstrou a importância de gerar bases de tempo previsíveis e assíncronas ao laço principal, evitando atrasos (delays) que prejudicam o funcionamento do código. A separação do cálculo numérico (feito em background na interrupção) do processo de atualização gráfica (multiplexação no loop infinito) resultou em um sistema estável, sem cintilação e com o brilho adequado dos LEDs.